DANIYA ALI

SEC D

CT-25159

MR.MUDASSIR ADMED

DSA LAB 5

QUESTION NO 1

SOURCE CODE:

Greedy > circular_queue.cpp

```cpp
#include <iostream>
#include <vector>
using namespace std;

class MyCircularDeque {
private:
    vector<int> arr;
    int front, rear, size, capacity;
public:
    MyCircularDeque(int k) {
        capacity = k;
        arr.resize(k);
        front = 0;
        rear = 0;
        size = 0;
    }

    bool insertFront(int value) {
        if (isFull()) return false;
        front = (front - 1 + capacity) % capacity;
        arr[front] = value;
        size++;
        return true;
    }

    bool insertLast(int value) {
        if (isFull()) return false;
        arr[rear] = value;
        rear = (rear + 1) % capacity;
        size++;
        return true;
    }

    bool deleteFront() {
        if (isEmpty()) return false;
        front = (front + 1) % capacity;
        size--;
        return true;
    }

    bool deleteLast() {
        if (isEmpty()) return false;
        rear = (rear - 1 + capacity) % capacity;
        size--;
        return true;
```

```
47
48      int getFront() {
49          if (isEmpty()) return -1;
50          return arr[front];
51      }
52
53      int getRear() {
54          if (isEmpty()) return -1;
55          return arr[(rear - 1 + capacity) % capacity];
56      }
57
58      bool isEmpty() {
59          return size == 0;
60      }
61
62      bool isFull() {
63          return size == capacity;
64      }
65  };
66
67  int main() {
68      MyCircularDeque myCircularDeque(3);
69      cout << myCircularDeque.insertLast(1) << endl;
70      cout << myCircularDeque.insertLast(2) << endl;
71      cout << myCircularDeque.insertFront(3) << endl;
72      cout << myCircularDeque.insertFront(4) << endl;
73      cout << myCircularDeque.getRear() << endl;
74      cout << myCircularDeque.isFull() << endl;
75      cout << myCircularDeque.deleteLast() << endl;
76      cout << myCircularDeque.insertFront(4) << endl;
77      cout << myCircularDeque.getFront() << endl;
78      return 0;
79  }
```

OUTPUT:

```
PS C:\Users\User\OneDrive\Desktop\DSA\Greedy> cd "c:\Us
1
1
1
0
2
1
1
1
4
PS C:\Users\User\OneDrive\Desktop\DSA\Greedy>
```

QUESTION NO 2

SOURCE CODE:

```cpp
#include <iostream>
#include <queue>
using namespace std;

int findTheWinner(int n, int k) {
    queue<int> q;
    for (int i = 1; i <= n; i++) {
        q.push(i);
    }
    while (q.size() > 1) {
        for (int i = 0; i < k - 1; i++) {
            q.push(q.front());
            q.pop();
        }
        q.pop(); // Eliminate the k-th friend
    }
    return q.front();
}

int main() {
    int n = 5, k = 2;
    cout << "Winner: " << findTheWinner(n, k) << endl;
    return 0;
}
```

OUTPUT:

```
4
PS C:\Users\User\OneDrive\Desktop\DSA\Greedy> cd "c:\Use
Winner: 3
PS C:\Users\User\OneDrive\Desktop\DSA\Greedy>
```

QUESTION NO 3

SOURCE CODE:

```cpp
#include <iostream>
using namespace std;

struct Call {
    int callId;
    string customerName;
};

class CallCenter {
private:
    queue<Call> callQueue;
    int totalCSRs;
    int availableCSRs;
public:
    CallCenter(int csrs) {
        totalCSRs = csrs;
        availableCSRs = csrs;
    }

    void addCall(int id, string name) {
        callQueue.push({id, name});
        cout << "Call #" << id << " from " << name << " added to queue." << endl;
    }

    void processCalls() {
        while (!callQueue.empty() && availableCSRs > 0) {
            Call nextCall = callQueue.front();
            callQueue.pop();
            availableCSRs--;
            cout << "CSR assigned to handle Call #" << nextCall.callId << " (" << nextCall.customerName << ")." << endl;
            // Simulate call completion freeing up CSR
            availableCSRs++;
        }
    }
};

int main() {
    CallCenter center(2);
    center.addCall(101, "Alice");
    center.addCall(102, "Bob");
    center.addCall(103, "Charlie");
    center.processCalls();
    return 0;
}
```

OUTPUT:

```
PS C:\Users\User\OneDrive\Desktop\DSA\Greedy> cd "
Call #101 from Alice added to queue.
Call #102 from Bob added to queue.
Call #103 from Charlie added to queue.
CSR assigned to handle Call #101 (Alice).
CSR assigned to handle Call #102 (Bob).
CSR assigned to handle Call #103 (Charlie).
PS C:\Users\User\OneDrive\Desktop\DSA\Greedy>
```

QUESTION NO 4

SOURCE CODE:

Greedy > product_of_stream.cpp

```cpp
#include <iostream>
#include <vector>
using namespace std;

class ProductOfNumbers {
private:
    vector<int> prefixProducts;
public:
    ProductOfNumbers() {
        prefixProducts.push_back(1);
    }

    void add(int num) {
        if (num == 0) {
            prefixProducts = {1};
        } else {
            prefixProducts.push_back(prefixProducts.back() * num);
        }
    }

    int getProduct(int k) {
        int n = prefixProducts.size();
        if (k >= n) return 0;
        return prefixProducts.back() / prefixProducts[n - 1 - k];
    }
};

int main() {
    ProductOfNumbers productOfNumbers;
    productOfNumbers.add(3);
    productOfNumbers.add(0);
    productOfNumbers.add(2);
    productOfNumbers.add(5);
    productOfNumbers.add(4);
    cout << productOfNumbers.getProduct(2) << endl;
    cout << productOfNumbers.getProduct(3) << endl;
    cout << productOfNumbers.getProduct(4) << endl;
    return 0;
}
```

OUTPUT:

```
PS C:\Users\User\OneDrive\Desktop\DSA\Greedy> cd "c:
20
40
0
PS C:\Users\User\OneDrive\Desktop\DSA\Greedy>
```

QUESTION NO 5

SOURCE CODE:

```cpp
class DataStream {
private:
    int targetValue;
    int k;
    int consecutiveCount;
public:
    DataStream(int value, int k) {
        this->targetValue = value;
        this->k = k;
        this->consecutiveCount = 0;
    }

    bool consec(int num) {
        if (num == targetValue) {
            consecutiveCount++;
        } else {
            consecutiveCount = 0;
        }
        return consecutiveCount >= k;
    }
};

int main() {
    DataStream dataStream(4, 3);
    cout << dataStream.consec(4) << endl;
    cout << dataStream.consec(4) << endl;
    cout << dataStream.consec(4) << endl;
    cout << dataStream.consec(3) << endl;
    return 0;
}
```

OUTPUT:

```
PS C:\Users\User\OneDrive\Desktop\DSA\Greedy> cd "c:\Users\
0
0
1
0
PS C:\Users\User\OneDrive\Desktop\DSA\Greedy>
```